

API Testing for QA Engineers

Page 1 - Introduction to API Testing

Modern applications rely heavily on APIs (Application Programming Interfaces). APIs allow systems such as web applications, mobile apps, payment gateways, and cloud services to communicate with each other.

API testing focuses on validating the functionality, reliability, performance, and security of these APIs.

Key goals of API testing:

- Verify correct data exchange
- Validate business logic
- Ensure backend reliability
- Detect defects early

Page 2 - What is an API

API stands for Application Programming Interface.

It acts as a bridge between two applications.

Communication flow:

Client → API → Server → API Response → Client

Example: A mobile app requesting user information from backend server.

Page 3 - Real Life API Example

Restaurant example:

- Customer → Waiter → Kitchen → Waiter → Customer
- Mapping:
 - Customer = Client
 - Waiter = API
 - Kitchen = Server



@testingvala



info@testingvala.com

The waiter transfers the request to the kitchen and returns the response.

Page 4 - API Communication Flow

Typical API communication process:

1. Client sends request
2. API receives request
3. Server processes request
4. Server sends response
5. Client receives response

Page 5 - Types of APIs

Common types of APIs:

- REST API
- SOAP API
- GraphQL API

REST APIs are the most widely used today because they are lightweight and scalable.

Page 6 - REST API Characteristics

REST APIs are:

- Stateless
- Fast
- Lightweight
- Scalable

They usually exchange data using JSON.

Example endpoint: <https://reqres.in/api/users>

Page 7 - HTTP Protocol

HTTP (HyperText Transfer Protocol) is used for communication between clients and servers.

Every API request uses HTTP methods.

📷 @testingvala

✉ info@testingvala.com

Page 8 - HTTP Methods

Common HTTP methods:

- GET - Retrieve data
- POST - Create data
- PUT - Update data
- PATCH - Partially update data
- DELETE - Remove data

Page 9 - GET Request

GET is used to retrieve information from the server.

Example:

GET <https://regres.in/api/users?page=2>

Response: List of users.

Page 10 - POST Request

POST is used to create new resources.

Example:

POST <https://regres.in/api/users>

Request body:

```
{ "name": "John", "job": "QA Engineer" }
```

Page 11 - PUT Request

PUT updates an entire resource.

Used when replacing existing data.

 @testingvala  info@testingvala.com

PATCH updates only specific fields.

Example:

PATCH <https://reqres.in/api/users/2>

Example change: updating job title.

Page 13 – DELETE Request

DELETE removes a resource from the system.

Example:

DELETE <https://reqres.in/api/users/2>

Expected status: 204 No Content.

Page 14 – API Request Structure

An API request contains:

- Endpoint
- Method
- Headers
- Body

Example endpoint:

<https://reqres.in/api/users>

Page 15 – Headers

Headers provide additional information about the request.

Common headers:

- Content-Type: application/json
 - Authorization: Bearer Token
-

Page 16 – Request Body

Body contains data sent to the server.

Example:


```
{ "name": "Jane", "job": "Tester" }
```

Page 17 - API Response

API responses include:

- Status Code
- Response Body
- Headers
- Response Time

Example status:

200 OK

Page 18 - Status Codes

Postman is a widely used API testing tool.

It allows users to:

- Send API requests
- View responses
- Organize collections
- Write tests

Page 20 - Introduction to Postman

Postman is a widely used API testing tool.

It allows users to:

- Send API requests
- View responses
- Organize collections
- Write tests

Page 20 - Creating a Request in Postman

Steps:

1. Open Postman
2. Select HTTP method
3. Enter API URL
4. Click Send

📷 @testingvala

✉ info@testingvala.com

Page 21 - Collections in Postman

Collections help organize requests.

Example structure:

User API Collection → Get Users → Create User → Update User → Delete User

Page 22 - API Validation

QA engineers validate:

- Status code
- Response data
- Headers
- Response time
- Data correctness

Page 23 - Best Practices

API testing best practices:

- Validate response schema
- Test positive and negative scenarios
- Validate status codes
- Automate repetitive tests

Page 24 - API Testing Tools

Common tools:

- Postman
- REST Assured
- Swagger
- SoapUI
- Insomnia

Page 25 - Conclusion

API testing is an essential skill for modern QA engineers.

Benefits:

- Faster testing
- Better backend validation
- Early defect detection
- Supports automation

Learning API testing improves both manual and automation testing capabilities.



@testingvala



info@testingvala.com